The Commit Is the Label: Git-Native, Self-Verifying Memory for AI Coding Agents

Cong Guo[1] ⟨*author list*⟩

[1]Rekal — rekal.dev · github.com/rekal-dev/rekal-cli

DRAFT v0.1 — architecture and benchmark design are final; all empirical values marked ⟨·⟩ are pending the corpus run of §5.

Abstract. AI coding agents start every session amnesic: the conversations in which a team weighed approaches, rejected alternatives, and decided are gone the moment the session ends, so agents re-propose what was already ruled out and re-litigate what was already settled. Recent memory systems answer with compiled knowledge structures — tiered stores, memory graphs, wikis — that must themselves be maintained, and with self-judged experience admission that is vulnerable to self-confirmation. We present **Rekal**, a git-native memory system for coding agents built on three design commitments that invert the prevailing recipe: (1) **preserve, don't compress** — the store is an append-only ledger of raw session turns, captured by a post-commit hook, with every derived structure (full-text, LSA, and neural-embedding indexes) disposable and rebuilt rather than maintained; (2) **the commit is the label** — each checkpoint links sessions to the commit they produced, giving free, objective supervision that dialogue-memory benchmarks must hand-annotate; and (3) **the merge is the verifier** — only sessions whose code landed on the default branch are shared to the team, an external admission gate that self-judged memory lacks. On these labels we build **RekalBench**, the first benchmark for repo-grounded intent recall: five task families (provenance, decision recall, dead-end awareness, multi-hop synthesis, decision drift) mined from a real corpus of ⟨*N*⟩ sessions / ⟨*N*⟩ turns, evaluated against a no-memory floor, a strong grep/direct-corpus-interaction baseline, static distilled notes, and single-signal ablations. Rekal's hybrid recall attains ⟨*MRR*⟩ versus ⟨*MRR*⟩ for the strongest baseline, reaches correct answers at ⟨*k*⟩× fewer context tokens, and holds retrieval quality flat as the corpus scales while unbounded grep degrades. All data stays on the operator's machine; the harness is public and self-serve.

1 Introduction

Code has a ledger; intent does not. Version control records every line a team ships, but the reasoning that produced those lines — explored designs, rejected alternatives, the correction a reviewer shouted mid-session — lives in AI-assistant transcripts that expire with the terminal window. The cost compounds as agents do more of the work: an agent that cannot remember what its own team already tried will confidently re-propose it.

The research community has converged on the diagnosis but not the cure. Long-horizon memory systems organize dialogue history into tiered stores [1], associative graphs [2], or compiled wikis [3], and report strong gains on conversational benchmarks such as LoCoMo [4]. Yet three structural problems recur across this literature. First, **derived structure goes stale**: systems that compile knowledge offline acknowledge freshness and maintenance of that structure as an open problem, and the more elaborate the structure, the heavier the liability [1], [2], [3], [5]. Second, **self-judged experience poisons itself**: when the same agent executes, summarizes, and admits its own memories, wrong-but-self-consistent trajectories are stored as successes and amplified on reuse — the self-confirmation trap [6]. Third, **evaluation lacks ground truth**: dialogue corpora have no objective notion of "the memory that mattered," so benchmarks depend on costly human annotation [4], [7].

This paper's observation is that the software-engineering setting dissolves all three problems at once, if the memory system is built **into git** rather than beside it:

- **Freshness.** Rekal's store is two databases with strictly separated roles: `data.db`, an append-only ledger of raw, scrubbed session turns — the only source of truth — and `index.db`, every derived structure (BM25 full-text, latent-semantic and neural embeddings, co-occurrence and lineage tables), **disposable by construction** and rebuilt from the ledger at any time. Derived structure that can be thrown away cannot go stale.
- **Supervision.** A post-commit hook checkpoints the active session(s) and records which sessions produced which commit. The commit is an objective, free, abundant label connecting a unit of intent to a verified unit of code — supervision that dialogue memory must manufacture by annotation.
- **Verification.** Rekal shares a session with the team only when its commit is reachable from the default branch (merge-commit, rebase, or patch-equivalent squash). Code review, CI, and the merge itself act as an **external** admission gate on shared memory — a stronger verifier than the model-consensus gates proposed to escape the self-confirmation trap [6] — while unmerged and abandoned work remains local as explicitly **negative** knowledge: the map of dead ends.

Contributions. (1) The design and implementation of Rekal, a local-first, git-native memory engine for coding agents (single binary; capture, storage, hybrid recall, and team sync with no server, API, or telemetry), together with an agent-facing skill layer that operationalizes active, multi-step memory reconstruction [2] over the engine's primitives. (2) **RekalBench**, the first benchmark for repo-grounded intent recall, whose ground truth is mined from the corpus's own commit–session structure rather than annotated. (3) An empirical study on a real working corpus (*⟨N sessions, N turns, N tool calls across N repositories⟩*) against the baselines that matter in practice — including the strongest current objection, direct corpus interaction over raw transcripts with shell tools [8] — with token-cost and corpus-scale analyses. (4) Ablations isolating each recall signal (lexical, latent-semantic, neural, steering boost, subagent down-weighting), enabled by query-time weighting that requires no reindexing.

2 Related Work

Long-horizon agent memory. AdaMem organizes dialogue into working/episodic/persona/graph tiers with question-conditioned routing and a write path protected from reads, reporting state-of-the-art LoCoMo F1 of 44.65 with GPT-4.1-mini [1]. MRAgent represents memory as a cue–tag–content graph and — centrally — replaces one-shot lookup with **active reconstruction**, proving a formal separation between active and passive retrieval and reporting a 23% relative multi-hop gain on LoCoMo with lower token cost than strong baselines [2]. EMem argues the opposite of aggressive consolidation: fine-grained, self-contained discourse units with normalized entities and **source-turn attribution** form a simple baseline that rivals elaborate systems [9]. Rekal takes EMem's side of the compression axis at the storage layer (raw turns, attributed snippets) and MRAgent's side of the retrieval axis at the interaction layer (skills drive iterative search–facet–zoom–drill loops over engine primitives).

Compiled versus query-time structure. LLM-Wiki compiles a corpus into bidirectionally linked pages traversed with search/read/follow tools, outperforming graph-RAG systems by 2.0–8.1 F1 on multi-hop QA [3], but inherits a compilation artifact that must be maintained. SAG instead instantiates relational structure **at query time** by SQL joins over flat event/entity indexes, making incremental update trivial [5]. Rekal's storage is architecturally aligned with SAG: DuckDB tables (full-text, embeddings, file co-occurrence, checkpoint links) with per-query weighting and joins, and no compiled artifact other than content-hash-cached embeddings.

Retrieval interfaces for agents. Direct corpus interaction (DCI) equips an agent with grep, file reads, and shell over the raw corpus — no index at all — and beats sparse, dense, and reranking retrieval on several benchmarks [8], with trained variants stronger still [10]. This is the serious objection to any index: transcripts are files; why not grep them? RISE supplies the answer at scale: unbounded shell interaction degrades sharply with corpus size — at one million documents DCI accuracy falls to 60% with a third of runs hitting wall-clock failure at roughly $1.10 per query, while a **bounded interaction space** constructed by retrieval holds 78% accuracy at $0.28 [11]. Rekal is precisely a bounded-interaction- space constructor for intent history: hybrid scoring bounds the space to a ranked set of sessions; windowed, role-filtered drilling explores inside it. §6 reproduces the crossover on session data.

Experience admission and memory integrity. EDV names the self-confirmation trap and decouples execution, distillation, and verification, gating memory admission on consensus [6]. Surveys of memory security catalogue write-path poisoning attacks and contamination propagation across sessions and agents [12], [13], and controlled experiments show sanitization is only effective **before** summarization, not after [14]. Rekal's answers are structural: the write path is the operator's own post-commit hook (no open write API); content is scrubbed **before** insertion and stored uncompressed; every memory carries provenance to a commit; and the shared tier admits only merge-verified experience. Governance concerns for enterprise memory [15] map onto git's existing controls (review, branch protection, ACLs).

Self-improvement from traces. RHO improves an agent harness from unlabeled past trajectories, accepting changes only by head-to-head self-preference over the incumbent [16]; AutoMem treats memory management itself as a trainable skill, revising the memory structure from trajectory review [17]; LRAT mines retrieval supervision from agent behavior rather than human labels [18]. Rekal's corpus natively contains all three ingredients — trajectories, their outcomes (commits, merges), and the agent's own recall behavior — and §7 sketches the resulting data flywheel. BeliefShift motivates our decision-drift task: memories are beliefs that get revised, and a memory system must surface the **current** decision [7].

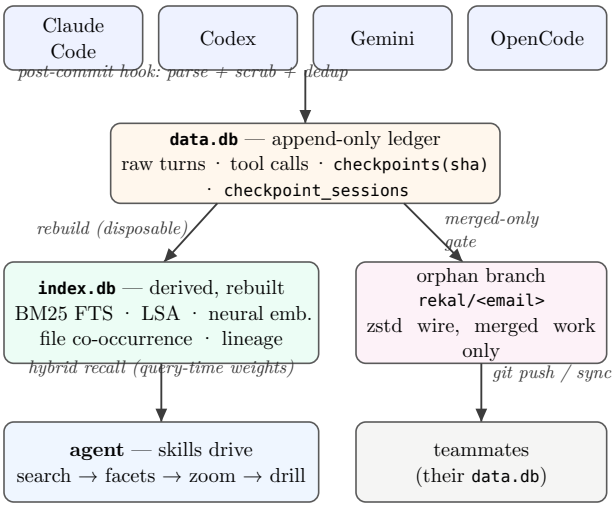

Figure 1: Rekal architecture. One source of truth (append-only `data.db`); all derived structure disposable (`index.db`); sharing gated on merge verification. No server, no API, no telemetry — git is the only wire.

3 Rekal: Design

3.1 Capture: preserve, don't compress

A post-commit hook parses the active assistant session(s) — adapters cover Claude Code, Codex, Gemini, and OpenCode — deduplicates turns, scrubs secrets and anonymizes paths **before any byte is stored** (the ordering shown to be the only safe one [14]), and appends to `data.db`. Nothing is summarized at write time: following the preservation result of [9], the unit of storage is the raw turn with role, order, timestamp, and tool-call structure, and every recall result carries `snippet_turn_index` attribution back to its source turn. Write-time compression is lossy and unfixable; query-time distillation improves with every model generation.

One class of distillation is nonetheless **harvested**: when the harness compacts a long session it writes an LLM summary of the conversation so far back into the transcript (10–17KB enumerating files touched, decisions, errors and fixes). Rekal captures these as turns with a dedicated role `summary` — already paid for, cumulative (the latest subsumes the rest), and never confused with human intent. Rekal generates no summaries of its own: a commit-time summary would be query-blind, written before any question exists, whereas the drill is query-aware and paid lazily. Rows stored before the role existed are reclassified in the derived views by a stable content fingerprint, scoped to the originating harness — the append-only ledger is never rewritten.

3.2 Storage: one ledger, disposable indexes

`data.db` is the only source of truth and is append-only. `index.db` holds everything derived: BM25 full-text over turns, latent-semantic (LSA) and neural (nomic-class) session embeddings, file-touch and co-occurrence tables, session lineage (parent/subagent), and facets. Any migration, corruption, or model upgrade is handled by deletion and rebuild; content-hash-keyed embedding caching makes rebuilds incremental. This is the structural answer to the maintenance problem that compiled-memory systems name and defer [1], [3], [5]: **the freshness of a structure you can throw away is not a research problem.**

3.3 Recall: a bounded interaction space

`rekal "<question>"` scores sessions by a weighted hybrid of BM25, LSA, and neural similarity, boosts turns where a human redirected the agent (`human_steering` — the moments decisions actually got made), boosts harvested compaction summaries by a smaller factor (dense anchors, but machine text must not outrank human intent at equal relevance), down-weights subagent chatter, and returns scored JSON with per-session snippets. Weights live in local config and apply at **query time** — changing them requires no reindex, which is also what makes single-signal ablations free (§5). Each result additionally carries `summary_turn_index` — a pointer to the session's latest compaction summary, never the payload itself: inlining 10–17KB per result would spend context before any question justified it. The agent then drills **inside** the bounded set: the pointed-at summary as the cheapest whole-session overview, windowed turn ranges, role filters, tool-call and file views, full transcript only as a last resort. In RISE's terms [11], search constructs the interaction space and the drill tools explore it; the skill layer (six shipped playbooks: base search, provenance, self-reflection, four-library distillation, exhaustive census, and wiki materialization — committed topic pages whose admission gate is code review, not a judge model) is the active-reconstruction policy [2] that sequences those primitives. The wiki playbook additionally exploits the harness's workflow decomposition: topic clusters are independent and one cluster's sessions fit one context, so generation fans out as one subagent per topic whose transcript lands back in the ledger with lineage (`parent_session_id`, `workflow_name`) — the cost of distillation is measurable **from the store it distilled**, and recall's subagent down-weight keeps the meta-work from crowding out what it recorded.

3.4 Sharing: the merge is the verifier

`rekal push` exports to a per-author git orphan branch only those checkpoints whose commit is reachable from the default branch, with patch-equivalence detection for squash merges; everything else waits, releasing automatically if its branch later lands. The shared tier therefore admits only experience that survived review, CI, and merge — an external verification gate, in contrast to self- or consensus-judged admission [6]. Unmerged and abandoned branches remain in the local ledger as labeled **negative** knowledge: RekalBench task T3 tests exactly whether a system can warn "we tried this; it didn't land."

3.5 Scopes: wide reads locally, narrow writes globally

Rekal's memory has three scopes with asymmetric permeability. The **repo ledger** is truth. The **machine-wide index** optionally folds in the operator's other repos' sessions (explicit opt-in) — index-only, origin-labeled, and structurally unpushable: an imported session has no checkpoint in this repo's ledger, so no code path can ever export it. The **team wire** admits merged work only. Knowledge crosses a scope boundary exclusively through a human-visible artifact: sessions reach the team when their commit merges, and cross-repo experience becomes committed text in a repo through exactly one channel — a wiki page, generated in an explicit cross-repo mode that labels every foreign citation with its origin, shipped as a PR whose body declares what is crossing. Machine-wide memory stores in the literature [1], [17] make the opposite choice: one pool, implicitly readable and writable everywhere — precisely the open write path the security analyses identify as the contamination and exfiltration surface [12], [14]. Rekal's rule compresses to: **wide reads locally, narrow writes globally — egress is always a diff someone approved.**

4 RekalBench

No benchmark exists for repo-grounded intent recall: conversational-memory suites (LoCoMo [4], LongMemEval, PERSONAMEM) evaluate chat-persona recall, and IR suites (BEIR, BRIGHT) evaluate document QA. RekalBench's defining property is that its ground truth is **mined, not annotated**: the corpus's own commit–session structure supplies labels.

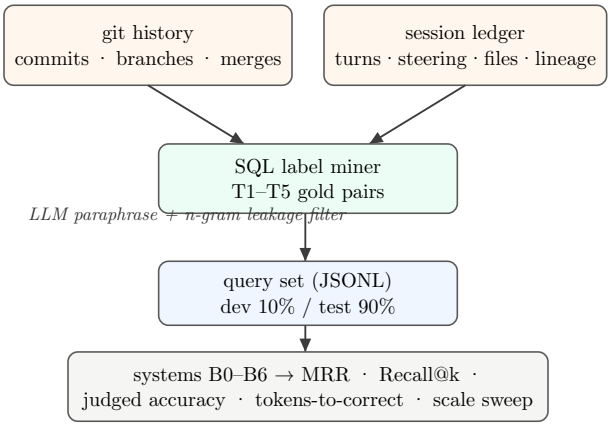

Figure 2: RekalBench pipeline. Labels are mined from structure the tool already records; paraphrase plus an n-gram overlap filter breaks lexical leakage between query and target.

4.1 Tasks

Task	Gold label source	Query generation	n
T1 Provenance	commit → producing session(s), via the `checkpoint_sessions` links	paraphrase of commit message + changed paths (not indexed content)	*⟨≈500⟩*
T2 Decision recall	`human_steering` turns	paraphrase of surrounding context; steering turn held out of prompt; 4-gram Jaccard ≤ 0.3	*⟨≈300⟩*
T3 Dead-end awareness	sessions on never-merged branches	"have we tried X?" from the branch's cumulative intent	*⟨≈50⟩*
T4 Multi-hop synthesis	session pairs linked by file co-occurrence / lineage	generator-validated two-session questions	*⟨≈100⟩*
T5 Decision drift	later session reversing an earlier decision on the same files (hand-confirmed sample)	"what is our current approach to X?"	*⟨≈30⟩*

Table 1: RekalBench task families. All labels derive from recorded structure; only T5 needs light manual confirmation.

Leakage controls. Commit messages are not part of indexed session text, so T1 queries are lexically de-correlated by construction; all tasks add LLM paraphrase with an n-gram overlap ceiling, and we report the residual overlap distribution. **Label noise.** A commit's linked sessions can include incidental chatter; we hand-audit 50 T1 pairs and report label precision (*⟨value⟩*). **Splits.** 10% dev (tuning allowed) / 90% test, one-shot — following the incumbent-versus-candidate discipline of [16].

5 Experimental Setup

5.1 Corpus

One operator's real working store, folded per-repo (labels require the repo's own checkpoint ledger) plus machine-wide for scale sweeps. The primary repo alone holds approximately 500 sessions, 63k turns, and 48k tool calls (preliminary counts; final corpus card: *⟨repos, sessions, turns, tool calls, steering turns, linked commits, date range⟩*). No session content leaves the machine; the paper reports aggregates only.

5.2 Systems

ID	System
B0	No memory — question (plus repo code where the task allows) only
B1	Grep/DCI [8] — same agent model with `rg/jq/sed` over raw transcript JSONL; GrepSeek-informed prompt [10]; equal turn budget
B2	Static notes — one-time LLM-distilled `MEMORY.md` (≤8k tokens), the folk practice
B3	BM25-only (Rekal weights `{1,0,0}`) — query-time ablation, no reindex
B4	Neural-only (weights `{0,0,1}`) — ablation
B5	Rekal full hybrid + steering boost + subagent down-weight
B6	Rekal + skills — B5 driven by the shipped playbooks (rungs 2–4)

Table 2: Systems under test.

5.3 Metrics

Rung 1 (retrievability): MRR, Recall@{1, 5, 10}, nDCG@10 with bootstrap CIs. Rung 2 (answer quality): LLM-judged correctness against the gold turn (distinct generate/answer/judge models; 50-sample human agreement check). Rung 3 (efficiency): context tokens loaded until first correct answer, wall-clock, and dollar cost — the axis RISE and MRAgent make primary [2], [11]. Rung 3 additionally includes a **judge-free drill-strategy proxy**: on queries where recall places the gold session in the top results, we compare the two drills an agent can make — a raw turn window around the matched turn versus the single turn `summary_turn_index` points at — on tokens ingested and gold-term coverage (fraction of the label's distinctive content words present in the drilled text). It measures context-assembly cost, not answer quality, and runs with no LLM in the loop.

The L3 gate (wiki experiment). Materializing a browsing layer — committed topic pages generated by the wiki playbook — is a cache of memory, and whether the cache is worth its cost is an empirical question, not an architectural one. We measure both sides. **Generation cost**, from the ledger itself: the workflow's subagent transcripts carry lineage, so turns/tool-calls per topic page are a SQL query over `session_facets`, and the tokens the run ingested are its harness accounting. **Cache payoff**, on broad queries (T1/T3-style): answering from the materialized page versus recall+drill, on tokens and gold-term coverage — the same proxy columns as the drill table. The static-notes baseline (B2) predicts the failure mode: pages age. We therefore also report the regeneration diff rate (pages invalidated per week of new sessions) as the cache's maintenance price. The diff is itself the observable: generation is deterministic (stable topic and edge ordering), so a re-run over unchanged history is an empty diff, and any non-empty diff **is** structural drift — an edge appearing in the index's topic graph is a correlation that did not exist last run, a removed edge is one that decayed, each transition reviewed before it is admitted. Mutating graph stores [1], [5] cannot show this drift to anyone; here `git log` over the index is a reviewed time series of the project's conceptual structure, and the maintenance problem is converted into code review. Finally, **cross-repo contribution**: pages generated twice, own-repo evidence only versus explicit cross-repo mode (§3.4′s reviewed-egress channel), report the fraction of pages with foreign citations and the broad-query coverage delta the foreign evidence buys — the value of machine-wide memory, measured at the only gate through which it can become shared. Scale sweep: metrics and B1 latency at 10/25/50/100% date-cut subsets. Freshness: recall bucketed by target-session age; index rebuild wall-clock versus corpus size. Rung 4 (agent-in-the-loop): 10–20 real tasks, A/B on human-steering count, re-proposal of known dead ends, and time-to-done.

6 Results

All values in this section are placeholders pending the corpus run; tables define the exact shape of the report.

6.1 Retrievability (rung 1)

System	T1 MRR	T1 R@5	T2 MRR	T3 R@5	T4 b@10
B1 grep/DCI	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩
B2 static notes	—	—	—	—	—
B3 BM25-only	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩
B4 neural-only	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩
B5 Rekal hybrid	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩	⟨·⟩

Table 3: Retrieval quality by task (test split, bootstrap 95% CIs). B2 has no per-query retrieval and is evaluated at rungs 2–3 only.

Planned analyses: per-signal contribution (B3/B4 vs B5), steering-boost on/off delta on T2, and label-precision-adjusted upper bounds.

6.2 Answer quality and token cost (rungs 2–3)

System	Judged acc.	Tokens → correct	$ / query
B0 no memory	⟨·⟩	—	—
B1 grep/DCI	⟨·⟩	⟨·⟩	⟨·⟩
B2 static notes	⟨·⟩	⟨·⟩	⟨·⟩
B5 Rekal	⟨·⟩	⟨·⟩	⟨·⟩
B6 Rekal + skills	⟨·⟩	⟨·⟩	⟨·⟩

Table 4: Answer quality and efficiency on a 200-query stratified subset. The headline claim has the shape: equal-or-better accuracy at ⟨*k*⟩× fewer tokens.

Drill strategy	Tokens	Coverage	Cov. / 1k tok
window (5-turn, at match)	⟨·⟩	⟨·⟩	⟨·⟩
summary-first (pointer)	⟨·⟩	⟨·⟩	⟨·⟩

Table 5: Judge-free drill-strategy proxy (`run_rung3.py`), paired on queries where recall reaches the gold session and a compaction summary exists. Hypothesis: the trade is question-shaped — summary-first buys broad coverage at a fixed 10–17KB price and should win "what happened here" queries (T1/T3); the window should win pointed lookups (T2). The corpus run decides.

6.3 The L3 gate: is a materialized browsing layer worth its cost?

Measure	Value	Source
pages generated / sessions cited per page	⟨·⟩	wiki-run PR
generation: turns + tool calls per page	⟨·⟩	ledger (`workflow_name` lineage)
generation: tokens ingested per page	⟨·⟩	harness accounting
broad-query answering: page vs recall+drill, tokens	⟨· *vs* ·⟩	proxy, drill-table columns
broad-query answering: page vs recall+drill, coverage	⟨· *vs* ·⟩	proxy, drill-table columns
regeneration diff rate (pages invalidated / week)	⟨·⟩	watermark vs new sessions
cross-repo mode: pages with foreign citations / coverage delta	⟨· / ·⟩	origin labels; A/B on generation mode

Table 6: The wiki experiment. A committed topic-page layer is a **cache of memory**: generation cost is measured from the ledger's own lineage records (the memory system accounts for its own distillation), payoff on broad queries against recall+drill, and the maintenance price as the rate at which new sessions invalidate pages — the static-notes failure mode (B2), made continuous and measurable. A cached in-index L3 layer is built only if this table says so.

6.4 Scale and freshness (the RISE crossover, C4)

We re-run rung 1 at date-cut corpus subsets. The hypothesis, transplanted from [11]: B1's accuracy and latency degrade with corpus size (broad shell scans over ⟨*N*⟩k turns), while Rekal's bounded space holds flat; index rebuild remains ⟨*seconds–minutes*⟩ and recall shows no decay with target-session age. Deliverables: accuracy-versus-corpus-size and tokens-versus-accuracy curves (Fig. ⟨*3*⟩, ⟨*4*⟩).

6.5 Agent-in-the-loop (rung 4)

⟨*10–20 tasks; steering interventions, dead-end re-proposals, time-to-done, with and without Rekal*⟩ — run last, only if rungs 1–3 hold.

7 Discussion and Limitations

What the numbers can and cannot say. Rung 1 is a retrievability proxy — a hit is not a useful answer; that is why rungs 2–4 exist. Self-labels are noisy (audited, reported). A single-operator corpus is a case study until replicated: the harness is public, fully local, and runnable by any Rekal user on their own store, which is the honest path to multi-corpus evidence. LoCoMo-style persona memory is out of scope by design.

The grep objection, taken seriously. DCI is strong [8] and our B1 is deliberately not a straw man (published prompt, equal model and budget). Our position is conditional, matching RISE [11]: below some corpus size, grep is fine — and Rekal still adds parsing, scrubbing, provenance, and team sync; above it, a bounded interaction space is the difference between answers and wall-clock failures. The crossover point on session data is an empirical output of this work, not an assumption.

The flywheel (future work). The corpus contains its own improvement signal: which recalled sessions an agent drills into after a search is an implicit relevance label [18], enabling private per-corpus weight tuning, accepted only on head-to-head wins over the incumbent configuration [16]; trajectory review can revise the skill layer itself [17]. Temporal belief maintenance (surfacing the **latest** decision, flagging reversals [7]) is a query-time view away (T5 measures it).

8 Conclusion

Memory for coding agents does not need another compiled structure to maintain or another self-judged experience store to poison itself. It needs a ledger that already has ground truth. Git supplies the label (the commit), the verifier (the merge), and the transport (the repo); Rekal supplies the capture, the disposable indexes, the bounded recall, and the agent-facing playbooks. RekalBench turns that same structure into the first benchmark for repo-grounded intent recall — and every claim in this paper is falsifiable by running it, locally, on your own history.

Reproducibility. Engine, skills, benchmark spec, extraction SQL, and this paper's source: github.com/rekal-dev/rekal-cli (`docs/research/`). All benchmark data remains on the operator's machine; published artifacts are aggregates, prompts, and code.

References

[1] S. Yan, J. Ni, L. Zheng, and others, "AdaMem: Adaptive User-Centric Memory for Long-Horizon Dialogue Agents." [Online]. Available: https://arxiv.org/abs/2603.16496

[2] S. Ji, Y. Li, and B. Hooi, "Memory is Reconstructed, Not Retrieved: Graph Memory for LLM Agents." [Online]. Available: https://arxiv.org/abs/2606.06036

[3] H. Ming, F. Li, X. Wu, W. Que, and others, "Retrieval as Reasoning: Self-Evolving Agent-Native Retrieval via LLM-Wiki." [Online]. Available: https://arxiv.org/abs/2605.25480

[4] A. Maharana and others, “Evaluating Very Long-Term Conversational Memory of LLM Agents.” [Online]. Available: https://arxiv.org/abs/2402.17753

[5] Y. Wu, J. Li, X. Liang, and others, “SAG: SQL-Retrieval Augmented Generation with Query-Time Dynamic Hyperedges.” [Online]. Available: https://arxiv.org/abs/2606.15971

[6] S. Zhu, Y. Qi, Y. Wang, and others, “Escaping the Self-Confirmation Trap: An Execute-Distill-Verify Paradigm for Agentic Experience Learning.” [Online]. Available: https://arxiv.org/abs/2606.24428

[7] BeliefShift authors, “BeliefShift: Benchmarking Temporal Belief Consistency and Opinion Drift in LLM Agents.” [Online]. Available: https://arxiv.org/abs/2603.23848

[8] Z. Li, H. Zhang, and others, “Beyond Semantic Similarity: Rethinking Retrieval for Agentic Search via Direct Corpus Interaction.” [Online]. Available: https://arxiv.org/abs/2605.05242

[9] S. Zhou and J. Han, “A Simple Yet Strong Baseline for Long-Term Conversational Memory of LLM Agents.” [Online]. Available: https://arxiv.org/abs/2511.17208

[10] GrepSeek authors, “Training Search Agents for Direct Corpus Interaction.” [Online]. Available: https://arxiv.org/abs/2605.29307

[11] S. Zhuang and others, “Towards Retrieving Interaction Spaces for Agentic Search.” [Online]. Available: https://arxiv.org/abs/2606.06880

[12] Survey authors, “A Survey on Long-Term Memory Security in LLM Agents: Attacks, Defenses, and Governance Across the Memory Lifecycle.” [Online]. Available: https://arxiv.org/abs/2604.16548

[13] Survey authors, “From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents.” [Online]. Available: https://arxiv.org/abs/2606.04990

[14] State-contamination authors, “State Contamination in Memory-Augmented LLM Agents.” [Online]. Available: https://arxiv.org/abs/2605.16746

[15] H. Taheri, “Governed Memory: A Production Architecture for Multi-Agent Workflows.” [Online]. Available: https://arxiv.org/abs/2603.17787

[16] W. Pan and others, “Evolving Agents in the Dark: Retrospective Harness Optimization via Self-Preference.” [Online]. Available: https://arxiv.org/abs/2606.05922

[17] S. Wu, H. Zhu, Y. Zhang, X. Wang, and S. Yeung-Levy, “AutoMem: Automated Learning of Memory as a Cognitive Skill.” [Online]. Available: https://arxiv.org/abs/2607.01224

[18] Y. Zhou and others, “Learning to Retrieve from Agent Trajectories.” [Online]. Available: https://arxiv.org/abs/2604.04949